

# HDOC DAY-27

**Title:** Pyramid/Diamond Pattern Generation using Parameterless Functions and Nested Loops

## 1. Problem Statement

Write a menu-driven C program using a switch statement to print various "Pyramid/Diamond patterns" selected by the user. Use nested loops without arrays. Each pattern activity must be implemented in a separate parameterless function returning void (void function\_name(void)):

1. **Full pyramid:** Input  $n$  and print a full pyramid using asterisks with  $2n - 1$  asterisks at base.
2. **Inverted pyramid:** Input  $n$  and print an inverted pyramid using asterisks.
3. **Diamond pattern:** Input  $n$  for upper half and print a diamond pattern of  $2n - 1$  rows with  $2n - 1$  asterisks at middle.
4. **Palindromic number pyramid:** Input  $n$ , print natural numbers increasing 1 to row number then decreasing back to 1.

5. **Repeated-number pyramid:** Input  $n$ , print current row number  $2n - 1$  times (where  $n$  denotes current row index).
6. **Exit:** Terminate program loop.

## 2. Algorithms

### Main Control Flow

1. Display Pyramid/Diamond Patterns Menu inside a do-while loop.
2. Read choice and route through switch statement to execute corresponding parameterless function.
3. Exit when choice 6 is entered.

### Pattern Functions

- **print\_full\_pyramid():** Read  $n$ . Loop  $i$  from 1 to  $n$ : print  $n - i$  double spaces, then  $2i - 1$  asterisks.
- **print\_inverted\_pyramid():** Read  $n$ . Loop  $i$  from  $n$  down to 1: print  $n - i$  double spaces, then  $2i - 1$  asterisks.
- **print\_diamond\_pattern():** Read  $n$ . Loop upper half ( $i = 1$  to  $n$ ), then lower half ( $i = n - 1$  down to 1), printing  $n - i$  spaces and  $2i - 1$  asterisks.
- **print\_palindromic\_number\_pyramid():** Read  $n$ . Loop  $i$  from 1 to  $n$ : print spaces, increasing numbers from 1 to  $i$ , then decreasing numbers from  $i - 1$  down to 1.

- **print\_repeated\_number\_pyramid():** Read  $n$ . Loop  $i$  from 1 to  $n$ : print  $n - i$  spaces, then print  $i$  value  $2i - 1$  times.

### 3. Test Case Table

| Test Case | Pattern / Option | Input n | Expected Pattern Output               | Actual Output           | Status |
|-----------|------------------|---------|---------------------------------------|-------------------------|--------|
| 1         | Full Pyramid     | $n = 4$ | 4 rows pyramid (base 7 stars)         | Matched expected output | Pass   |
| 2         | Inverted Pyramid | $n = 4$ | 4 rows inverted pyramid (top 7 stars) | Matched expected output | Pass   |
| 3         | Diamond Pattern  | $n = 4$ | 7 rows diamond (middle 7 stars)       | Matched expected output | Pass   |

| Test Case | Pattern / Option           | Input n    | Expected Pattern Output              | Actual Output           | Status |
|-----------|----------------------------|------------|--------------------------------------|-------------------------|--------|
| 4         | Palindromic Number Pyramid | $n = 4$    | Rows:<br>1/121/<br>12321/<br>1234321 | Matched expected output | Pass   |
| 5         | Repeated -Number Pyramid   | $n = 4$    | Rows:<br>1/222/<br>33333/<br>4444444 | Matched expected output | Pass   |
| 6         | Invalid Choice             | Choice = 9 | Error message displayed              | Error message shown     | Pass   |
| 7         | Exit Option                | Choice = 6 | "Exiting program ..."                | Program exited          | Pass   |

## 4. Source Code Listing

```
#include <stdio.h>

void print_full_pyramid(void) {
    int n;
    printf("\n--- Full Pyramid ---\n");
    printf("Enter number of rows (n): ");
    scanf("%d", &n);
    if (n <= 0) { printf("Error: Input must be a positive integer.\n"); return; }
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= (2 * i - 1); j++) printf("* ");
        printf("\n");
    }
}

void print_inverted_pyramid(void) {
    int n;
    printf("\n--- Inverted Pyramid ---\n");
    printf("Enter number of rows (n): ");
    scanf("%d", &n);
    if (n <= 0) { printf("Error: Input must be a positive integer.\n"); return; }
    for (int i = n; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= (2 * i - 1); j++) printf("* ");
        printf("\n");
    }
}

void print_diamond_pattern(void) {
    int n;
    printf("\n--- Diamond Pattern ---\n");
    printf("Enter number of rows for upper half (n): ");
    scanf("%d", &n);
    if (n <= 0) { printf("Error: Input must be a positive integer.\n"); return; }
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= (2 * i - 1); j++) printf("* ");
        printf("\n");
    }
}
```

```

    }
    for (int i = n - 1; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= (2 * i - 1); j++) printf("* ");
        printf("\n");
    }
}

void print_palindromic_number_pyramid(void) {
    int n;
    printf("\n--- Palindromic Number Pyramid ---\n");
    printf("Enter number of rows (n): ");
    scanf("%d", &n);
    if (n <= 0) { printf("Error: Input must be a positive integer.\n"); return; }
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= i; j++) printf("%d ", j);
        for (int j = i - 1; j >= 1; j--) printf("%d ", j);
        printf("\n");
    }
}

void print_repeated_number_pyramid(void) {
    int n;
    printf("\n--- Repeated-Number Pyramid ---\n");
    printf("Enter number of rows (n): ");
    scanf("%d", &n);
    if (n <= 0) { printf("Error: Input must be a positive integer.\n"); return; }
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= (2 * i - 1); j++) printf("%d ", i);
        printf("\n");
    }
}

int main(void) {
    int choice;
    do {
        printf("\n===== PYRAMID / DIAMOND PATTERNS MENU\n");

```

```
printf("1. Full pyramid\n");
printf("2. Inverted pyramid\n");
printf("3. Diamond pattern\n");
printf("4. Palindromic number pyramid\n");
printf("5. Repeated-number pyramid\n");
printf("6. Exit\n");
printf("Enter choice (1-6): ");
scanf("%d", &choice);

switch (choice) {
    case 1: print_full_pyramid(); break;
    case 2: print_inverted_pyramid(); break;
    case 3: print_diamond_pattern(); break;
    case 4: print_palindromic_number_pyramid(); break;
    case 5: print_repeated_number_pyramid(); break;
    case 6: printf("Exiting program...\n"); break;
    default: printf("Error: Invalid choice!\n"); break;
}
} while (choice != 6);
return 0;
}
```

#### 4. Execution & Verification Screenshots

```

(kali@kali)-[~]
└─$ mkdir HDOC_Day27
cd HDOC_Day27
(kali@kali)-[~/HDOC_Day27]
└─$ nano day27.c
(kali@kali)-[~/HDOC_Day27]
└─$ gcc day27.c -o day27
(kali@kali)-[~/HDOC_Day27]
└─$ ./day27

===== PYRAMID / DIAMOND PATTERNS MENU =====
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter choice (1-6): 1

— Full Pyramid —
Enter number of rows (n): 4
  *
 ***
*****
*****
*****
*****
*****
*****

===== MAIN MENU =====
===== PYRAMID / DIAMOND PATTERNS MENU =====
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter choice (1-6): 2

— Inverted Pyramid —
Enter number of rows (n): 4
*****
*****
*****
*****
*****
*****
*****
*****

===== SPECIAL NUMBER PATTERNS SUBMENU =====
===== PYRAMID / DIAMOND PATTERNS MENU =====
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter choice (1-6): 3

— Diamond Pattern —
Enter number of rows for upper half (n): 4
  *
 ***
*****
*****
*****
*****
  *

===== MAIN MENU =====
===== PYRAMID / DIAMOND PATTERNS MENU =====
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter choice (1-6): 4

— Palindromic Number Pyramid —
Enter number of rows (n): 4
  1
 1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1

===== SPECIAL NUMBER PATTERNS SUBMENU =====
===== PYRAMID / DIAMOND PATTERNS MENU =====
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter choice (1-6): 5

— Repeated-Number Pyramid —
Enter number of rows (n): 4
  1
 2 2 2
3 3 3 3 3
4 4 4 4 4 4 4

===== PYRAMID / DIAMOND PATTERNS MENU =====
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter choice (1-6): 6
Exiting program...

===== MAIN MENU =====
(kali@kali)-[~/HDOC_Day27]
└─$

```